Phần 1: Thư viện & Định nghĩa lớp Student, Node

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.Comparator;
import java.util.List;

// 1. Lớp Student
class Student {
    int id;
    String name;
    String gender;
    double result;

    public Student(int id, String name, String gender, double result) {
        this.id = id;
        this.name = name;
        this.gender = gender;
        this.result = result;
    }

    @Override
    public String toString() {
        return String.format("| %-6d | %-20s | %-8s | %-6.1f |", id, name, gender, result);
    }
}

// Node của cây BST
class Node {
    Student data;
    Node left;
    Node right;

    public Node(Student data) {
        this.data = data;
        this.left = null;
        this.right = null;
    }
}
```

Phần 2: Cấu trúc StudentBST - Hàm chèn (insert) & Duyệt bảng (printTable)

```
// 2. Cây Binary Search Tree (BST)
class StudentBST {
    Node root;

    public StudentBST() {
        this.root = null;
    }

    // Chèn node theo ID
    public void insert(Student s) {
        root = insertRec(root, s);
    }

    private Node insertRec(Node root, Student s) {
        if (root == null) {
            return new Node(s);
        }
        if (s.id < root.data.id) {
            root.left = insertRec(root.left, s);
        } else if (s.id > root.data.id) {
            root.right = insertRec(root.right, s);
        }
        return root;
    }

    // Duyệt In-order (trái -> gốc -> phải) để xuất danh sách tăng dần theo ID
    public void printTable() {
        System.out.println("+--------+----------------------+----------+--------+");
        System.out.println("| ID     | Ho va Ten            | Gioi Tinh| Diem   |");
        System.out.println("+--------+----------------------+----------+--------+");
        inOrderPrint(root);
        System.out.println("+--------+----------------------+----------+--------+");
    }

    private void inOrderPrint(Node root) {
        if (root != null) {
            inOrderPrint(root.left);
            System.out.println(root.data);
            inOrderPrint(root.right);
        }
    }
```

Phần 3: Phương thức toList, Tìm kiếm (search) & Bắt đầu hàm Xóa (delete)

```
    // Lấy tất cả sinh viên vào 1 danh sách hỗ trợ tính toán
    public List<Student> toList() {
        List<Student> list = new ArrayList<>();
        toListRec(root, list);
        return list;
    }

    private void toListRec(Node root, List<Student> list) {
        if (root != null) {
            toListRec(root.left, list);
            list.add(root.data);
            toListRec(root.right, list);
        }
    }

    // Tìm kiếm theo ID
    public Student search(int id) {
        Node curr = root;
        while (curr != null) {
            if (id == curr.data.id) return curr.data;
            if (id < curr.data.id) curr = curr.left;
            else curr = curr.right;
        }
        return null;
    }

    // Xóa theo ID
    public void delete(int id) {
        root = deleteRec(root, id);
    }

    private Node deleteRec(Node root, int id) {
        if (root == null) return null;
        if (id < root.data.id) {
            root.left = deleteRec(root.left, id);
        } else if (id > root.data.id) {
            root.right = deleteRec(root.right, id);
        } else {
```

Phần 4: Xử lý xóa node 2 con, toàn bộ hàm minValueNode & Đóng class StudentBST

```
            // Node có 0 hoặc 1 con
            if (root.left == null) return root.right;
            if (root.right == null) return root.left;

            // Node có 2 con: tìm node nhỏ nhất bên cây con phải
            Node minNode = minValueNode(root.right);
            root.data = minNode.data;
            root.right = deleteRec(root.right, minNode.data.id);
        }
        return root;
    }

    private Node minValueNode(Node node) {
        Node current = node;
        while (current.left != null) {
            current = current.left;
        }
        return current;
    }
}
```

Phần 5: Lớp Main - Thực thi Yêu cầu 1 (Khởi tạo mảng 10 sinh viên & chèn vào BST)

```
// 3. Hàm Main thực thi các yêu cầu
public class Main {
    public static void main(String[] args) {
        StudentBST bst = new StudentBST();

        // ================= YÊU CẦU 1 =================
        System.out.println("========== YEU CAU 1: KHOI TAO & XUAT DANH SACH BST THEO ID ======
====");
        Student[] students = {
            new Student(104, "Le Van Minh", "nam", 6.0),
            new Student(106, "Do Phuong Mai", "nu", 8.5),
            new Student(102, "Le Thi Hoang Anh", "nu", 2.5),
            new Student(107, "Pham Tuan Hai", "nam", 4.0),
            new Student(101, "Tran Van Tra", "nam", 9.0),
            new Student(103, "Le Vu Long", "nam", 3.0),
            new Student(108, "Phan Thi Yen", "nu", 7.0),
            new Student(109, "Le Van Chau", "nam", 6.5),
            new Student(112, "Pham Tra My", "nu", 7.5),
            new Student(105, "Dong Van Trang", "nam", 5.5)
        };

        for (Student s : students) {
            bst.insert(s);
        }
        bst.printTable();
```

Phần 6: Thực thi Yêu cầu 2 (Tính tổng SV, điểm TB, tỷ lệ xếp loại, Max/Min)

```
        // ================= YÊU CẦU 2 =================
        System.out.println("\n========== YEU CAU 2: THONG KE & TIM KIEM DIEM ==========");
        List<Student> list = bst.toList();
        int total = list.size();
        System.out.println("1. Tong so sinh vien: " + total);

        double sum = 0;
        int gioi = 0, kha = 0, tb = 0, yeu = 0;
        Student maxStudent = list.get(0);
        Student minStudent = list.get(0);

        for (Student s : list) {
            sum += s.result;
            // Xếp loại học lực
            if (s.result >= 8.0) gioi++;
            else if (s.result >= 7.0) kha++;
            else if (s.result >= 5.0) tb++;
            else yeu++;

            // Điểm cao nhất / thấp nhất
            if (s.result > maxStudent.result) maxStudent = s;
            if (s.result < minStudent.result) minStudent = s;
        }

        System.out.printf("2. Diem trung binh cua sinh vien: %.2f\n", (sum / total));
        System.out.println("3. Ti le xep loai sinh vien:");
        System.out.printf("   - Gioi (>= 8.0)      : %d/%d (%.1f%%)\n", gioi, total, (gioi * 1
00.0 / total));
        System.out.printf("   - Kha  (>= 7.0)      : %d/%d (%.1f%%)\n", kha, total, (kha * 10
0.0 / total));
        System.out.printf("   - Trung binh (>= 5.0): %d/%d (%.1f%%)\n", tb, total, (tb * 100.0
/ total));
        System.out.printf("   - Yeu  (< 5.0)       : %d/%d (%.1f%%)\n", yeu, total, (yeu * 10
0.0 / total));
        System.out.println("4. Sinh vien diem cao nhat: " + maxStudent.name + " (" + maxStuden
t.result + " diem)");
        System.out.println("   Sinh vien diem thap nhat: " + minStudent.name + " (" + minStude
nt.result + " diem)");
```

Phần 7: Thực thi Yêu cầu 3 & Trích lọc, sắp xếp theo điểm (Yêu cầu 4)

```
        // ================= YÊU CẦU 3 =================
        System.out.println("\n========== YEU CAU 3: TIM KIEM & XOA THEO ID ==========");
        int searchId = 106;
        Student found = bst.search(searchId);
        System.out.println("1. Tim sinh vien co ID = " + searchId + ":");
        if (found != null) {
            System.out.println("   Tim thay: " + found);
        } else {
            System.out.println("   Khong tim thay sinh vien co ID = " + searchId);
        }

        int deleteId = 103;
        System.out.println("\n2. Xoa sinh vien co ID = " + deleteId + " khoi danh sach:");
        bst.delete(deleteId);
        System.out.println("Danh sach sau khi xoa:");
        bst.printTable();

        // ================= YÊU CẦU 4 =================
        System.out.println("\n========== YEU CAU 4: TRICH XUAT & SAP XEP DANH SACH =========
=");
        List<Student> currentList = bst.toList();
        List<Student> dsDat = new ArrayList<>();
        List<Student> dsRot = new ArrayList<>();

        for (Student s : currentList) {
            if (s.result >= 5.0) {
                dsDat.add(s);
            } else {
                dsRot.add(s);
            }
        }

        System.out.println("--- Danh sach sinh vien DAT (>= 5.0) ---");
        xuatDanhSach(dsDat);
        System.out.println("\n--- Danh sach sinh vien ROT (< 5.0) ---");
        xuatDanhSach(dsRot);

        // Sắp xếp theo điểm thi tăng dần
        System.out.println("\n--- 1. Sap xep danh sach DAT theo diem thi (tang dan) ---");
        dsDat.sort(Comparator.comparingDouble(s -> s.result));
        xuatDanhSach(dsDat);

        System.out.println("\n--- Sap xep danh sach ROT theo diem thi (tang dan) ---");
        dsRot.sort(Comparator.comparingDouble(s -> s.result));
        xuatDanhSach(dsRot);
```

Phần 8: Sắp xếp theo tên, các hàm phụ trợ & Kết thúc chương trình

```
        // Sắp xếp theo tên sinh viên (theo từ cuối cùng của tên)
        Comparator<Student> nameComparator = (s1, s2) -> {
            String ten1 = layTen(s1.name);
            String ten2 = layTen(s2.name);
            return ten1.compareToIgnoreCase(ten2);
        };

        System.out.println("\n--- 2. Sap xep danh sach DAT theo ten ---");
        dsDat.sort(nameComparator);
        xuatDanhSach(dsDat);

        System.out.println("\n--- Sap xep danh sach ROT theo ten ---");
        dsRot.sort(nameComparator);
        xuatDanhSach(dsRot);
    }

    // Hàm phụ trợ in danh sách
    private static void xuatDanhSach(List<Student> list) {
        System.out.println("+--------+----------------------+----------+--------+");
        System.out.println("| ID     | Ho va Ten            | Gioi Tinh| Diem   |");
        System.out.println("+--------+----------------------+----------+--------+");
        for (Student s : list) {
            System.out.println(s);
        }
        System.out.println("+--------+----------------------+----------+--------+");
    }

    // Hàm phụ trợ tách lấy tên chính (từ cuối cùng)
    private static String layTen(String fullName) {
        String trimmed = fullName.trim();
        int lastSpace = trimmed.lastIndexOf(" ");
        if (lastSpace != -1) {
            return trimmed.substring(lastSpace + 1);
        }
        return trimmed;
    }
}
```